Week 8: Motion Detection

Laboratory 5

Last updated July 25, 2023

00. Content

Mathematics

- N/A

Programming Skills

- Functions and Modules

Embedded Systems

- Thonny and Micropython

0. Required Hardware

- Raspberry Pi Pico
- Breadboard
- USB connector
- Camera (Arducam HM01B0)
- 8 Wires

Write your name and email below:

Name: Yoon Kim

Email: kim4169@purdue.edu

You should work in groups of 2-3 for this lab.

Write your groupmates' names below:

Groupmate(s): Esha Garg, Yun-Jen Wu

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

1. Background Subtraction

There is a popular image processing technique called background subtraction. The most effective way to utilize background subtraction for motion detection is when:

1. the camera is stationary
2. the background is static
3. the lighting conditions do not change
4. there is minimal noise in the image

One scenario where all of these conditions are met is in production/manufacturing lines. In controlled indoor environments, long-term changes are rare.

Essentially, background subtraction allows for the separation of the background from the foreground in an image. For instance, if a camera is set up to monitor a doorway, most of the time, nothing is in motion within the frame. By selecting an initial reference frame, we can compare all subsequent frames to the reference frame.

In [2]:
```
vid = cv2.VideoCapture('test_vid.mov')
height = vid.get(cv2.CAP_PROP_FRAME_HEIGHT)
width  = vid.get(cv2.CAP_PROP_FRAME_WIDTH)
scale = 0.125                                          # smaller scale for fa
new_size = (int(width*scale),int(height*scale))
frames = []                                            # saving frames to a l
                                                       # methods quickly with
while vid.isOpened():
    success, frame = vid.read()
    if not success:
        print("Unable to read frame. Exiting ...")
        break
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    frame = cv2.resize(frame,dsize=new_size)
    frames.append(frame)
    cv2.imshow('frame', frame)                         # display grayscaled v
    if cv2.waitKey(25) == ord('q'):                    # press Q on keyboard
        break
vid.release()
cv2.destroyAllWindows()
```

Background subtraction refers to a whole class of methods. However, for the scope of this lab, we will only be using one specific method. To demonstrate an example, we will utilize the same test video from the previous lab. To keep things concise, we won't delve into the detailed workings of `cv2.createBackgroundSubtractorMOG2()`. Generally, this method is slightly more robust against illumination changes. The output of the background subtraction function is known as a *mask*, which indicates that **the output is a binary image**. Pixel values in the mask are either 0 (representing the background) or 255 (representing the foreground).

Run the cell to observe the results!

In [3]:
```
fgbg = cv2.createBackgroundSubtractorMOG2(detectShadows=False)      # define t

for f in frames:
    img = fgbg.apply(f)
    cv2.imshow('background subtracted frame',img)
    if cv2.waitKey(25) == ord('q'):                                 # press Q
        break
cv2.destroyAllWindows()
```

2. Motion Detection and Localization

Exercise 1

Suppose we apply background subtraction to each video frame and divide the frame into an $M \times N$ grid, resulting in a total of MN blocks. Can you consider a (mathematical) rule to determine whether there is movement in a specific block?

Hint: Would the proportion of certain grayscale values be high or low when there is motion?

Write Answers for Exercise 1 Below

The proportion of higher grayscale values will be higher.

When the frame is divided into an M×N grid, each block will have a subset of these pixels

3. Connecting the Camera

This time, we will record our own videos using the Arducam HM01B0, which is a small camera that can be connected to the Pico.

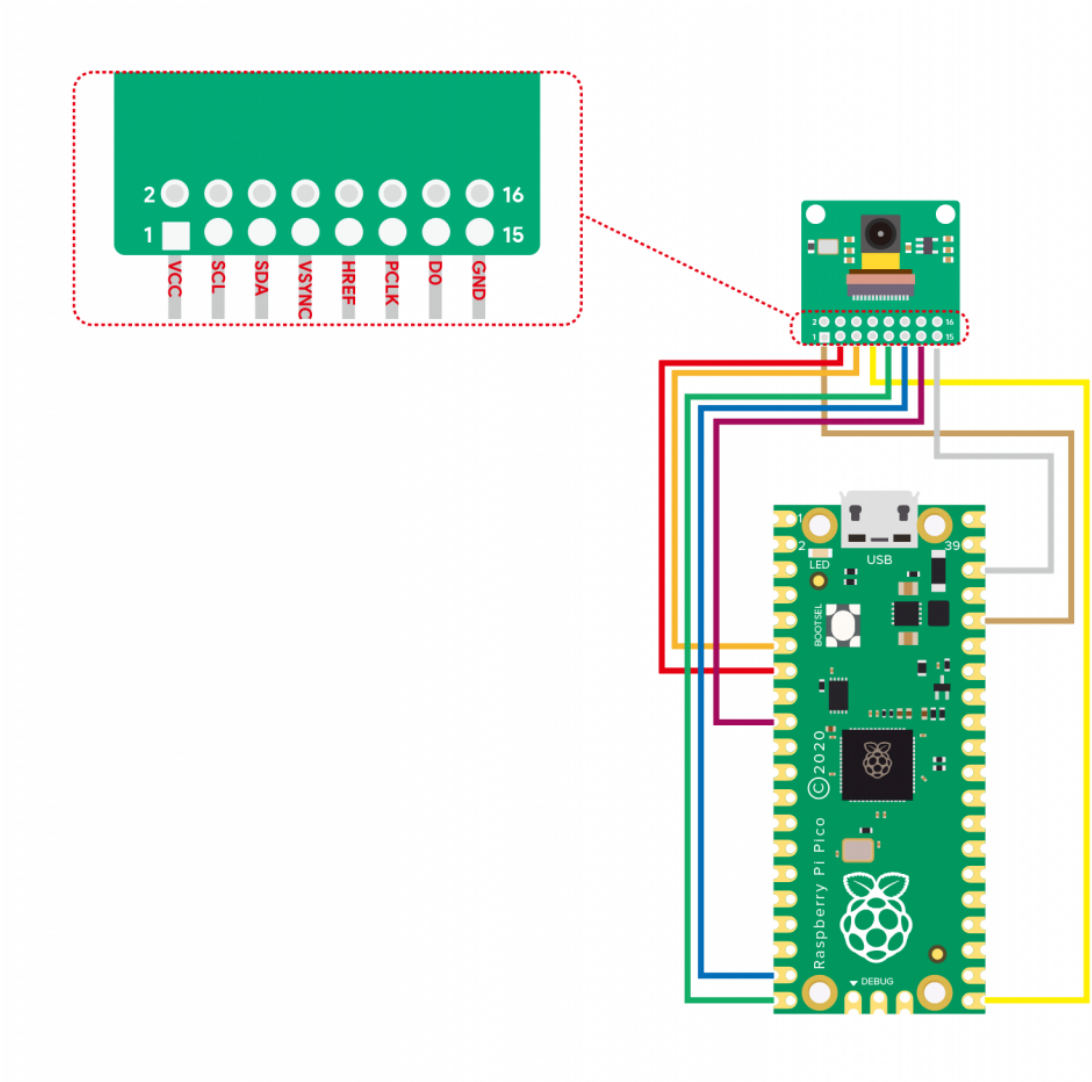
Wiring Instructions

Please ensure that your microcontroller is not connected to the computer while you are wiring components together. If you are unsure about your wiring, please consult the instructor. Use your jumper wires to establish the following connections:

HM01B0	Pico
VCC	3V3
SCL	GP5
SDA	GP4
VSYNC	GP16
HREF	GP15
PCLK	GP14

HM01B0	Pico
DO	GP6
GND	GND

Here is an image of the completed breadboard:



To find the names of the pins on the Raspberry Pi Pico, you can refer to its pinout diagram located here or in the Extra Materials section. The HM01B0, on the other hand, should have its pins labeled.

After confirming that the wiring is correct, press and hold the BOOTSEL button on the Pico while plugging it in. Download the arducam.uf2 file and copy it onto the Pico's drive using your computer's file manager (it should be listed as an external drive: "RPI-RP2") and not with Thonny. Once the file transfer is complete, the Pico will automatically disconnect, and its LED will start blinking rapidly.

Once the Pico has been successfully connected, please execute the following cell to ensure that we have successfully detected the Pico.

Exercise 2

Using your camera, locate the area of motion in the video frame in real time (or near real time). You can employ any method to accomplish this, whether it involves background subtraction, your solution to the previous exercise, a combination of the two, or any other approach you can think of. Determine how to visually indicate the detected movement on the camera, such as printing a statement in Python or drawing a rectangle on the frame.

Display all the code you used in a cell below.

Note: If you encounter significant hardware issues, you can try applying your method to the test video instead.

Write Answers for Exercise 2 Below

In [4]:
```
import time
import serial
from serial.tools import list_ports

PICO_HWID = "2E8A"


def get_pico_port():
    pico_ports = list(list_ports.grep(PICO_HWID))
    if len(pico_ports) == 0:
        raise Exception(
            "No Raspberry Pi Pico was detected. Check to make sure it is plugg
        )
    return pico_ports[0].device


print("Here are all the serial devices detected:")
for port in list_ports.comports():
    print(port.device, port.hwid)

port = get_pico_port()
print(f"\nselected port {port} as most likely to have a raspberry pi pico")
```

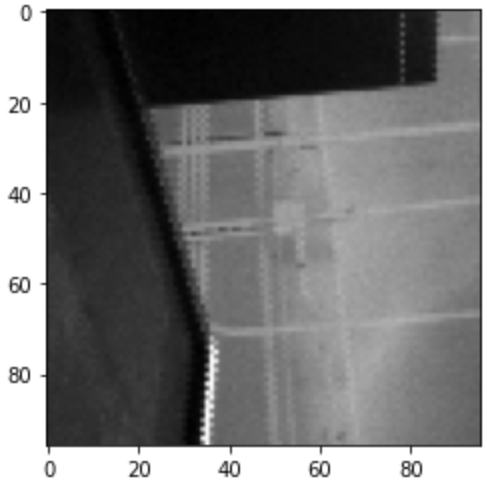
```
Here are all the serial devices detected:
/dev/ttyS1 n/a
/dev/ttyS0 PNP0501
/dev/ttyACM0 USB VID:PID=2E8A:000A SER=E66118604B7D5121 LOCATION=1-1.3:1.0

selected port /dev/ttyACM0 as most likely to have a raspberry pi pico
```

In [5]:
```
buffer = bytearray(96 * 96)
img = np.zeros(shape=(96, 96), dtype="uint8")

with serial.Serial(port, timeout=1) as s:
    s.read_until(b"\x55\xAA")
    s.readinto(buffer)
    img.flat[::-1] = buffer
```

```
plt.imshow(img, cmap="gray")
plt.show()
```

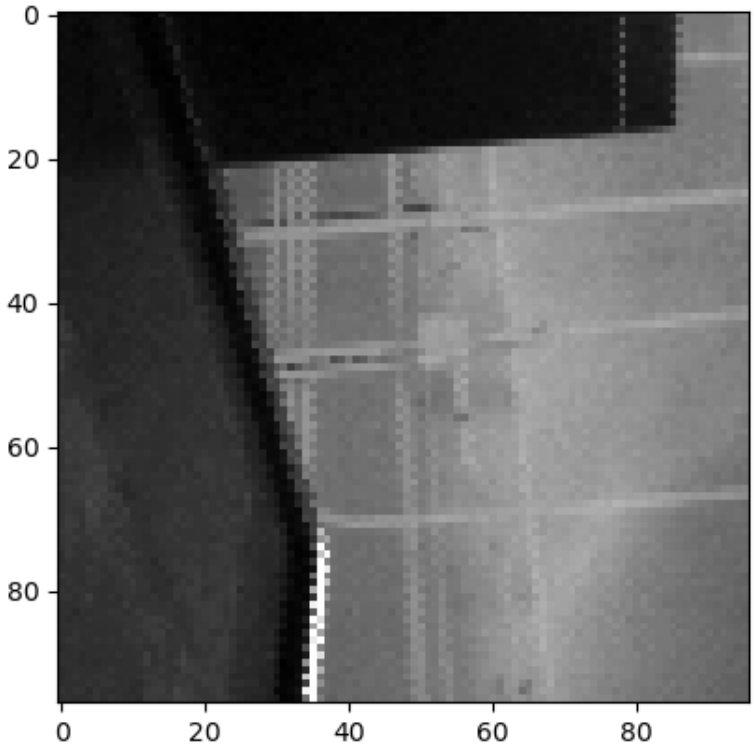


In [6]:

```
import time
import serial
from serial.tools import list_ports

%matplotlib widget
fig, ax = plt.subplots()
render = ax.imshow(img, cmap='gray')
plt.show(block=False)
```

Figure



In [7]:
```
fgbg = cv2.createBackgroundSubtractorMOG2(detectShadows=False)
```

In [9]:
```
frames = [];

try:
    with serial.Serial(port, timeout=1) as s:
        while True:
            s.read_until(b"\x55\xAA")
            s.readinto(buffer)
            img.flat[::-1] = buffer
            render.set_data(img)
            fig.canvas.draw()
            frames.append(img);
except KeyboardInterrupt:
    pass
```

In [11]:
```
def check_illum(quarter):
    quarter = quarter.flatten()
    count = 0
    for q in quarter:
        if q > 128:
            count = count + 1
        if count > 0.25* 47 * 47:
            return True
```

In [12]:
```
quarters = {"q1": [0,47,0,47], "q2":[0,47, 47,95],
            "q3":[47,95, 0,47],
            "q4":[47,95, 47,95]}

quarter_list = ["q1", "q2", "q3", "q4"]
quarter_list
try:
    with serial.Serial(port, timeout=1) as s:
        while True:
            s.read_until(b"\x55\xAA")
            s.readinto(buffer)
            img.flat[::-1] = buffer
            img = fgbg.apply(img)

            for i in range(len(quarter_list)):
                cord = quarters[quarter_list[i]]
                quarter = img[cord[0]:cord[1], cord[2]:cord[3]]

                if check_illum(quarter):
                    cv2.rectangle(img, (cord[0], cord[2]), (cord[1], cord[3]),
            render.set_data(img)
            fig.canvas.draw()
except KeyboardInterrupt:
    pass
```

```
---------------------------------------------------------------------------
SerialException                           Traceback (most recent call last)
Input In [12], in <cell line: 7>()
      8 with serial.Serial(port, timeout=1) as s:
      9     while True:
---> 10         s.read_until(b"\x55\xAA")
     11         s.readinto(buffer)
     12         img.flat[::-1] = buffer

File /home/shay/c/ma19000/miniconda3/envs/science/lib/python3.10/site-package
s/serial/serialutil.py:663, in SerialBase.read_until(self, expected, size)
    661 timeout = Timeout(self._timeout)
    662 while True:
--> 663     c = self.read(1)
    664     if c:
    665         line += c

File /home/shay/c/ma19000/miniconda3/envs/science/lib/python3.10/site-package
s/serial/serialposix.py:595, in Serial.read(self, size)
    588 else:
    589     # read should always return some data as select reported it was
    590     # ready to read when we get to this point.
    591     if not buf:
    592         # Disconnected devices, at least on Linux, show the
    593         # behavior that they are always ready to read immediately
    594         # but reading returns nothing.
--> 595         raise SerialException(
    596             'device reports readiness to read but returned no data '
    597             '(device disconnected or multiple access on port?)')
    598     read.extend(buf)
    600 if timeout.expired():

SerialException: device reports readiness to read but returned no data (device
disconnected or multiple access on port?)
```

Exercise 3

Provide a description of how effectively the motion detector/locator works in a paragraph or two. What methods did your group try? What challenges arose, and were you able to resolve any of them?

Write Answers for Exercise 3 Below

We divided the frame in four quarters and then looped through the quarter list, if there's motion detected it prints rectangle.

Reflection

Who would win in a fight? A billion lions or all the pokemon? Discuss with your group.

All the pokemon would win

In [ ]:

```

```